

Homework-5: Spark Machine Learning

Deadline: October 23rd, 11:59 PM ET

In this homework, you are expected to practice spark machine learning library.

Your homework submission should be on GitHub. Use the following GitHub classroom to access the assignment and create your assignment repository:

<https://classroom.github.com/a/1GvkHPC>

- **Don't create a public repository in your own GitHub account. Use the URL above to create the GitHub repository for this assignment. It includes a starter ReadMe file.**
- On Gradescope, only submit a URL to your GitHub Classroom repository.
- **All screenshots must include your full name in a text editor.**
 - **If your output is not printed in a Jupyter notebook, take a screenshot of the output and make sure your name is written in a Text Editor in the screenshot (Don't photoshop your name).**
- Not following the submission guidelines may result in grading penalties.

Cite any external sources you use. External sources shouldn't exceed more than 30% of the final solution.

Submit the HW in one or more Jupyter notebooks (or a series of Python files with clear comments). If you need to include scripts outside Jupyter for a given question, note it in your notebook. If you need to include output or screenshots, you may include them in your notebook or attach them as separate image files using the naming convention qx.png (where x is the question number). If you have more than one image for a given question, name the files as qx_1.png, qx_2.png, etc.

Q1. (20%) SparkML builds on the concept of Transformers, Estimators, and Pipelines. In Lecture 10, we created a pipeline for data engineering and preprocessing, but the pipeline does not include machine learning models.

In this question, please build a **single Pipeline** that conducts not just data engineering and preprocessing as in the lecture, but also machine learning (including hyper-parameter tuning) using the logistic regression model. You may use the same data engineering stages as in the lecture, and you may choose your own hyper-parameter grid.

Fit your pipeline to the **raw training dataframe (loaded from file)** and then **use the fitted pipeline to transform the raw test dataframe**. **Print the schema and the first few rows of the transformed test dataframe.**

Note that **only one single pipeline** should be created. The input to the fitted pipeline should be the raw dataframe loaded from file, and the output should contain predictions made by the tuned model. In your submission, **include a screenshot of your code that shows all the stages in the Pipeline you created.**

why should i give ss if code ?

Q2. (60%) Multi-Class Classification. In this question, you will use Spark ML to predict not just whether there is an attack, but also the type of attack. In other words, this is a multi-class classification problem with 5 possible categories (normal, DOS, R2L, U2R, probing)

- Q2-1 (10%) Create a preprocess pipeline that conducts the usual data engineering steps (as in the lecture), and also create an outcome column that indicates which of the 5 categories the record belongs to. Apply the pipeline to the raw train and test data loaded from file, **print the schema and the first few rows** of the transformed train/test dataframe.
- Q2-2 (15%) Select 2 machine learning models. For each of the machine learning models, train it on the training dataset. Calculate and **print the train and test accuracy**. Calculate and **show the confusion matrix** for the predictions on the test dataset. **NOTE:** Please use a scalable approach for this question. Here, scalable means you should not call any method (e.g. the **toPandas()** method) that *collects* an entire dataframe with the same length as the test dataset to the driver program.
- Q2-3 (15%) For each of the 2 machine learning models, identify at least one hyper parameter, build a parameter grid, and conduct hyper-parameter tuning using cross-validation, with accuracy as the metric. Calculate and **print the test accuracy** after tuning. (Hint: for evaluating accuracy, you may need to find the appropriate Evaluator with the appropriate metric name).
- Q2-4 (5%) In your own words, explain why you chose the two machine learning models, and for each of the two models, why you chose the hyper-parameter(s) to tune, and how you designed the parameter grid. Also, please include a discussion on the comparison of the two models.
- Q2-5(10%) Pick one of the two ML models and run the training process (don't need to do hyper-parameter tuning) on the google cloud DataProc cluster in *cluster* mode. Show a **screenshot of the spark history server**, including the Jobs page and the Executor page.
- Q2-6 (5%) Continuing from Q2-5, on the DataProc cluster, pick a dataframe and **show how many partitions the underlying RDD has.**

Q3. (20%) RDD Basics. In this question, you are expected to practice your knowledge of basic RDD transformations and actions. **Only use RDD transformations/actions (e.g. map, reduce, etc).** Do not use dataframes. In completing this question, **your last RDD action should directly produce the desired result:** the counts of a user-specified list of words for Q3-1 and the top-k

frequent words for Q3-2. Do not attempt to first use RDD to calculate the counts of all words and then use non-RDD methods to calculate the desired result.

- Q3-1 (10%): This question is based on the word count example in Lecture 12 that calculates the counts of words for all words present in the text, which may be slow if there are a lot of distinct words in the text. In this question, you are required to write a function that uses a series of RDD transformations/actions to calculate *the counts of a user-specified list of words*.

Input to the function:

- `rdd_lines`: RDD[str] — each record is a text line containing whitespace-separated words.
- `target_list`: List[str] — the user-specified list of words to count.

Output: A list of tuples (word, count).

In your submission, include the following:

- A brief explanation (5-10 sentences) of what each of your RDD transformation/action does, and why they can achieve the desired outcome.
- Download text file from <https://www.guannanqu.com/files/cmutoolchain/sherlock.txt>, create a RDD using `rdd_lines = sc.textFile("sherlock.txt")`, set `target_list = ["sherlock", "holmes", "watson", "lestrade", "moriarty"]`. Apply your function and **print out the output of your function**.
- Q3-2 (10%): Similar to Q3-1, but now write a function that uses a series of RDD transformations/actions to calculate *the counts of the k most frequently occurring words*, where k is a user-specified value. The first input of this function is `rdd_lines` (same as Q3-1), and the second input of this function is an integer k.

In your submission, include:

- A brief explanation (5-10 sentences) of what each of your RDD transformation/action does, and why they can achieve the desired outcome.
- Apply your function to the same text file as Q3-1 with k=50, and **print out the output**.